HTTP connections in Flutter

From `http.get()` to testable clients

Key ideas

- **Request**: method, URL, headers, optional body. **Response**: status code, headers, body (a JSON *string*).
- HTTP is **stateless**: each request carries its own context (token, cookie).
- Testable by design: **inject the client, isolate parsing, model errors.**

```
GET /posts/1 HTTP/1.1
Host: jsonplaceholder.typicode.com
Accept: application/json

HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8
{"userId": 1, "id": 1, "title": "...", "body": ↩
    "..."}
```

Methods and status codes

Method	Idempotent	Purpose
`GET`	yes	read
`POST`	no	create, submit
`PUT`	yes	replace the whole resource
`PATCH`	not guaranteed	change some fields
`DELETE`	yes	remove

Idempotent: sending twice = sending once (safe to retry).

200 OK, 201 Created, 204 No Content	success (2xx)
400/422, 401, 403, 404, 429	client side (4xx)
500, 503	server side (5xx)

A DNS failure or no network gives **no** status at all: an exception.

http or dio?

`http`	Dart team, small, every platform; simple REST, learning
`dio`	`BaseOptions`, interceptors, `CancelToken`, `FormData`, progress; auth pipelines, uploads

```
final client = http.Client();
try {
  final res = await client.get(Uri.https(
      'jsonplaceholder.typicode.com',
      '/albums/1'));
} finally {
  client.close();   // release connections
}
final dio = Dio(BaseOptions(
  baseUrl: 'https://api.example.com',
  connectTimeout: const Duration(seconds: 10),
  receiveTimeout: const Duration(seconds: 30)));
```

Retry and HTTP/2 for Dio are add-on packages. No dependency "just in case".

Platform setup

```
flutter pub add http          # or dio
flutter pub add --dev mockito build_runner
```

```
<!-- android/app/src/main/AndroidManifest.xml -->
<uses-permission
  android:name="android.permission.INTERNET"/>
```

- Android: release builds need `INTERNET` in the *main* manifest.
- iOS/macOS: HTTPS (ATS); macOS needs the `network.client` entitlement.
- Web: the server must allow your origin (CORS).
- Emulator → your PC: `10.0.2.2`, not `localhost`.
- Local plain HTTP: `NSAllowsLocalNetworking` or a debug-only network config, never "allow arbitrary loads".

Model and GET

```
class Post {
  const Post({required this.id,
            required this.title});
  final int id;
  final String title;
  factory Post.fromJson(Map<String, dynamic> j)
    => Post(id: j['id'] as int,
            title: j['title'] as String);
}

Future<List<Post>> fetchPosts(
    http.Client client) async {
  final url = Uri.https(
      'jsonplaceholder.typicode.com', '/posts');
  final res = await client.get(url)
      .timeout(const Duration(seconds: 8));
  if (res.statusCode != 200) {
    throw HttpFailure(res.statusCode);
  }
  final data = jsonDecode(res.body);
  if (data is! List) {
    throw const FormatException(
        'Expected a JSON list');
  }
  return data.map((e) => Post.fromJson(
      e as Map<String, dynamic>)).toList();
}
```

- `Uri.https(host, path, query)`: encodes the URL; no concatenation.
- Check the status, *then* decode, *then* check the shape.
- No raw `Map` in widgets: transport and transformation stay apart.

Send data

```
const json = {'Content-Type':
    'application/json; charset=UTF-8'};
final res = await client.post(
  Uri.https('jsonplaceholder.typicode.com',
            '/posts'),
  headers: json,
  body: jsonEncode({'title': title,
                    'body': body, 'userId': 1}),
);
if (res.statusCode != 201) {
  throw HttpFailure(res.statusCode);
}
await client.put(uri, headers: json,
  body: jsonEncode(wholePost));
await client.patch(uri, headers: json,
  body: jsonEncode({'title': 'Only this'}));
final del = await client.delete(uri);
// success: 200 or 204 (empty body)
```

`jsonEncode` the map: a Dart `Map` is not JSON. A 201 from JSONPlaceholder proves an acknowledgement, not storage (`GET /posts/101` gives 404).

FutureBuilder: four states

```
late Future<List<Post>> _posts;
@override
void initState() {
  super.initState();


  _posts = widget.service.fetchPosts();
}
void _retry() => setState(() =>
    _posts = widget.service.fetchPosts());

FutureBuilder<List<Post>>(
  future: _posts,
  builder: (context, snapshot) {
    if (snapshot.connectionState !=
        ConnectionState.done) {
      return const Center(
          child: CircularProgressIndicator());
    }
    if (snapshot.hasError) {
      return RetryView(onRetry: _retry);
    }
    final posts = snapshot.data!;
    if (posts.isEmpty) {
      return const Text('No posts yet');
    }
    return ListView.builder(...);
  },
)
```

Create the Future once (`initState` or Retry), **never in** `build`: each rebuild would send a new request. An empty list is not an error.

Layers

UI → State → Repository → API service

API service: requests → models or typed failures. Repository: network or cache. UI: renders state. Each arrow is an injection point for tests.

Failures and what the user sees

Failure	Show
No connection (`ClientException`)	offline message + Retry
`TimeoutException`	“Taking too long” + Retry
400 / 422	the server’s message next to the field
401	session expired: back to login
403 / 404	not allowed / not found; no Retry
429	wait (`Retry-After`)
500 / 503	“try again later”, back-off
Bad JSON (`FormatException`, `TypeError`)	generic error; log a bug

`SocketException` is `dart:io` only (not web). A timeout stops waiting; it may not cancel the I/O.

```
} on DioException catch (e) {
  switch (e.type) {
    case DioExceptionType.connectionTimeout:
    case DioExceptionType.receiveTimeout:
      throw const NetworkTimeout();
    case DioExceptionType.badResponse:
      throw HttpFailure(e.response!.statusCode!);
    case DioExceptionType.cancel:
      throw const RequestCancelled();
    default:
      throw const NetworkUnavailable();
  }
}
```

Break it on purpose

`no-such-host.invalid`	exception, no status
`httpbin.org/status/404`	a real 404 response
`httpbin.org/status/503`	a real 503 response
`httpbin.org/delay/3` + 2 s timeout	`TimeoutException`
Wi-Fi off	`ClientException`

Testing pairs

Code under test + a **controlled** network boundary.

Pair	Fidelity / use
Public sandbox	highest, slowest: smoke tests
`shelf` local server	real sockets, headers, timeouts
`http_mock_adapter`	Dio pipeline and interceptors
`MockClient` / Mockito	fastest: unit tests, edge cases

MockClient (package:http/testing.dart)

```
test('returns Album on 200', () async {
  final client = MockClient((request) async {
    expect(request.url.path, '/albums/1');
    return http.Response(
      '{"id": 1, "title": "Test Album"}', 200);
  });
  final album = await fetchAlbum(client);
  expect(album.title, 'Test Album');
});
test('throws on 404', () async {
  final client = MockClient(
      (_) async => http.Response('', 404));
  await expectLater(fetchAlbum(client),
      throwsA(isA<HttpFailure>()));
});
```

Mockito (generated mocks)

```
@GenerateMocks([http.Client])
void main() {
  test('bad JSON', () async {
    final client = MockClient();
    when(client.get(any)).thenAnswer((_) async
        => http.Response('not json', 200));
    await expectLater(fetchAlbum(client),
        throwsA(isA<FormatException>()));
    verify(client.get(any)).called(1);
  });
}
```

```
dart run build_runner build
```

Use `any`, not `any()`. Hand-written mocks fail under null safety.

Dio adapter and a local server

```
final dio = Dio(BaseOptions(
    baseUrl: 'https://example.com'));
final adapter = DioAdapter(dio: dio);
adapter.onGet('/status',
  (server) => server.reply(200, {'message': 'ok'}))↩
    ;
expect((await dio.get('/status'))
    .data['message'], 'ok');
```

Mock the adapter, not `Dio`: interceptors still run.

```
setUp(() async {
  server = await io.serve((Request req) =>
    req.url.path == 'albums/1'
      ? Response.ok('{"id":1,"title":"Local"}',
          headers: {'content-type':
                    'application/json'})
      : Response.notFound(''),
    InternetAddress.loopbackIPv4, 0); // free port
});
tearDown(() => server.close(force: true));
// base URL: http://127.0.0.1:${server.port}
```

`shelf` runs in VM `test()`. In `testWidgets` real HTTP is blocked (400): inject a `MockClient`.

Sandboxes

JSONPlaceholder	`/posts`, `/albums`: all methods, writes simulated
httpbin / httpbun	`/headers`, `/status/404`, `/delay/3`
DummyJSON	`/products/search?q=phone`, `?limit=10&skip=10`
ReqRes	`/api/users?page=2`; login 400
PokeAPI	deeply nested JSON
OpenWeatherMap	API key: 401 bad key, 429 limit

Writes do not persist; services can be slow or down; never send real credentials; keep them out of fast tests.

Test pyramid

Base	`MockClient`/Mockito, parsing: every save
Middle	adapter mocks, `shelf`, widget tests with fake clients: CI
Top	live sandbox smoke tests: nightly

Do and avoid

- Do: inject clients; close them; timeouts everywhere; parse outside widgets; base URLs in configuration.
- Avoid: calling a live-API test a “unit test”; mocking `Dio` itself; trusting fake CRUD writes; catching `DioException` without its `type`; `http.get` inside `build`.